Mathematical_Foundations_for_Machine_Learning_(AIMLZC416)

September 2, 2026

Assignment 1

Name: Viradiya Tushar Rajeshbhai

Sign: વિરડીયા તુષાર રાજેશ.

Q1) Finding solutions of linear systems

1. Write a code taking as input a matrix A of size $m \times n$ and a vector b of size $m \times 1$, where m and n are arbitrarily large numbers and $m < n$, constructing the augmented matrix and performing

- **REF, and**
- **RREF**

without using any built-in functions. In case you encounter any division by 0, you can choose a different A and/or b. Note that this part should be explicitly there in the code and give a comment line on the same.

(1 mark + 1 mark)

[1]:
```
import random

# Setting a fixed seed ensures our randomly generated floating-point numbers
# remain consistent and reproducible every time the notebook is evaluated.
random.seed(42)

def generate_random_float_matrix(rows, cols, low=1.0, high=9.0):
    """
    Generates a matrix of the specified dimensions populated with random floats.
    The results are rounded to exactly 8 decimal places to comply with the␣
 ↪assignment instructions.
    """
    return [
        [round(random.uniform(low, high) + random.random(), 8) for _ in␣
 ↪range(cols)]
        for _ in range(rows)
```


વિરડીયા તુષાર રાજ.

```
    ]

def generate_random_float_vector(size, low=1.0, high=9.0):
    """
    Generates a column vector populated with random 8-decimal floats.
    """
    return [round(random.uniform(low, high) + random.random(), 8) for _ in␣
 ↪range(size)]

def create_augmented_matrix(A, b):
    """
    Constructs the augmented matrix by appending the vector b to the right side␣
 ↪of matrix A.
    The dimensions are calculated dynamically from the inputs to support␣
 ↪arbitrarily large matrices.
    """
    m = len(A)

    # Verify that the vector has the exact same number of rows as the matrix.
    if len(b) != m:
        raise ValueError(f"Dimension mismatch: Matrix A has {m} rows, but␣
 ↪Vector b has {len(b)} elements.")

    return [A[i][:] + [b[i]] for i in range(m)]

def compute_ref(matrix, is_augmented=True, tol=1e-10):
    """
    Transforms a matrix into Row Echelon Form (REF) using forward elimination.
    The is_augmented flag dynamically determines if the last column should be␣
 ↪treated as a constants vector.
    """
    # Create a deep copy of the matrix so the original input remains unmodified.
    M = [row[:] for row in matrix]
    m = len(M)

    # If the matrix is augmented, we do not look for pivots in the final␣
 ↪constants column.
    n = len(M[0]) - 1 if is_augmented else len(M[0])

    pivot_row = 0

    # Traverse column by column to establish our pivots.
    for col in range(n):
        if pivot_row >= m:
            break
```


Priyanka Kumar Dhar.

```
        # Use partial pivoting to find the row with the largest absolute value␣
↪in the current column,
        # which provides better numerical stability.
        max_row = pivot_row
        max_val = abs(M[pivot_row][col])
        for r in range(pivot_row + 1, m):
            if abs(M[r][col]) > max_val:
                max_val = abs(M[r][col])
                max_row = r

        # Following Prof. Saurabh's guidance, we handle potential division by␣
↪zero here.
        # If the largest value in the column is essentially zero, it means the␣
↪whole column
        # below is zero and there is no valid row to swap with. Instead of␣
↪regenerating matrices,
        # we skip this column so it becomes a free variable naturally.
        if max_val < tol:
            continue

        # Swap the working row with the row containing our optimal pivot␣
↪element.
        M[pivot_row], M[max_row] = M[max_row], M[pivot_row]

        # Perform forward elimination to zero out all elements below the pivot.
        for r in range(pivot_row + 1, m):

            # An extra safety check to prevent zero division before calculating␣
↪the factor.
            if abs(M[pivot_row][col]) < tol:
                continue

            factor = M[r][col] / M[pivot_row][col]
            M[r][col] = 0.0

            for c in range(col + 1, len(M[0])):
                M[r][c] -= factor * M[pivot_row][c]

        pivot_row += 1

    return M

def compute_rref(ref_mat, tol=1e-10):
    """
    Transforms a given REF matrix into Reduced Row Echelon Form (RREF) using␣
↪backward elimination.
```

```
    """
    M = [row[:] for row in ref_mat]
    m = len(M)
    n = len(M[0]) - 1

    # Iterate backwards from the bottom row up to the top.
    for r in range(m - 1, -1, -1):

        # Locate the leading non-zero element to act as our pivot for this row.
        pivot_col = -1
        for c in range(n):
            if abs(M[r][c]) > tol:
                pivot_col = c
                break

        # If we encounter a row of all zeros, safely ignore it.
        if pivot_col == -1:
            continue

        pivot_val = M[r][pivot_col]

        # Before scaling the row to make the leading entry 1.0, we must ensure
        # the pivot is strictly non-zero to avoid breaking the algorithm.
        if abs(pivot_val) < tol:
            continue

        # Scale the row by dividing every element by the pivot value.
        for c in range(pivot_col, len(M[0])):
            M[r][c] /= pivot_val

        # Eliminate all non-zero entries directly above our current pivot.
        for above_r in range(r):
            factor = M[above_r][pivot_col]
            for c in range(pivot_col, len(M[0])):
                M[above_r][c] -= factor * M[r][c]

    return M

print("The functions for creating the augmented matrix, REF, and RREF have been␣
 ↪loaded successfully.")
```

```
The functions for creating the augmented matrix, REF, and RREF have been loaded
successfully.
```

2. Write a Python code to identify the pivot and non-pivot columns and find the particular solution and solutions to $Ax = 0$.

(1 mark)

[2]:
```
def extract_system_solutions(rref_mat, tol=1e-10):
    """
    Analyzes the RREF matrix dynamically to classify columns and formulate
    both the particular solution and the homogeneous null space basis vectors.
    """
    m = len(rref_mat)
    n = len(rref_mat[0]) - 1

    pivot_columns = []
    pivot_row_map = {}

    # Scan the matrix to identify the pivot columns, which contain the basic variables.
    for r in range(m):
        for c in range(n):
            if abs(rref_mat[r][c] - 1.0) < tol:
                # Verify that this is indeed the first non-zero entry in the row.
                is_leading = True
                for prev_c in range(c):
                    if abs(rref_mat[r][prev_c]) > tol:
                        is_leading = False
                        break

                # If confirmed, record the column index and its corresponding row.
                if is_leading and c not in pivot_columns:
                    pivot_columns.append(c)
                    pivot_row_map[c] = r
                    break

    # Any column that is not a pivot column is a free variable column.
    free_columns = [c for c in range(n) if c not in pivot_columns]

    # Find the particular solution by setting all free variables to zero.
    # This leaves the pivot variables directly equal to the constants in the augmented column.
    x_particular = [0.0] * n
    for p_col in pivot_columns:
        r = pivot_row_map[p_col]
        x_particular[p_col] = rref_mat[r][n]

    # To find the solutions to Ax = 0, we find the basis vectors for the null space.
    # Do this by iterating through each free variable, setting it to 1, and the others to 0.
    null_basis = []
```


રાવરીયા કુમાર એમ.

```
    for free_c in free_columns:
        v_h = [0.0] * n
        v_h[free_c] = 1.0

        # Calculate the required values for the pivot variables to balance the␣
 ↪equation to zero.
        for p_col in pivot_columns:
            r = pivot_row_map[p_col]
            v_h[p_col] = -rref_mat[r][free_c]

        null_basis.append(v_h)

    return pivot_columns, free_columns, x_particular, null_basis

print("The function to extract pivot columns and solutions has been loaded␣
 ↪successfully.")
```

```
The function to extract pivot columns and solutions has been loaded
successfully.
```

3. Consider a random 5×7 matrix A and a suitable b and show the REF, RREF, pivot columns, non-pivot columns, the particular solution, the solutions to $Ax = 0$, the general solution and verify the general solution.

(1/4 × 8 = 2 marks)

[3]:
```
def display_matrix(mat, title):
    """A helper function to cleanly print matrices with the required 8-decimal␣
 ↪precision."""
    print(f"\n{title}")
    for row in mat:
        print("  [ " + ", ".join(f"{val:12.8f}" for val in row) + " ]")

# Define the dimensions dynamically as requested by the assignment.
m_size, n_size = 5, 7

# Use a separate seed for the demonstration data to ensure consistency.
random.seed(2026)

# Step 1: Generate the random float matrix A and vector b.
A_matrix = generate_random_float_matrix(m_size, n_size)
b_vector = generate_random_float_vector(m_size)

# Step 2: Construct the augmented matrix.
aug_matrix = create_augmented_matrix(A_matrix, b_vector)

# Step 3: Calculate the Row Echelon Form.
ref_matrix = compute_ref(aug_matrix)
```

```
# Step 4: Calculate the Reduced Row Echelon Form.
rref_matrix = compute_rref(ref_matrix)

# Step 5, 6, and 7: Extract the column classifications and system solutions.
p_cols, f_cols, x_p, null_vectors = extract_system_solutions(rref_matrix)

# Step 8: Formulate the general solution by applying arbitrary scalars to our␣
 ↪free variables.
# The general solution takes the form: x_general = x_p + c1*v1 + c2*v2
scalars = [3.14159265, -2.71828182]
x_general = x_p[:]

for i, vector in enumerate(null_vectors):
    for j in range(len(x_general)):
        # To avoid index errors if the number of free variables changes, we␣
 ↪safely apply scalars
        if i < len(scalars):
            x_general[j] += scalars[i] * vector[j]
        else:
            x_general[j] += 1.0 * vector[j]

# To verify the solution without using built-in functions, implement manual␣
 ↪matrix-vector multiplication.
def manual_matrix_vector_mult(mat, vec):
    return [sum(mat[r][c] * vec[c] for c in range(len(vec))) for r in␣
 ↪range(len(mat))]

Ax_calculated = manual_matrix_vector_mult(A_matrix, x_general)

# Determine the maximum residual error to confirm the accuracy of the␣
 ↪calculation.
max_residual = max(abs(Ax_calculated[i] - b_vector[i]) for i in␣
 ↪range(len(b_vector)))

# Printing all the required outputs dynamically
print("Question 1 Part 3: Numerical Demonstration and Verification Outputs")

display_matrix(A_matrix, f"Output 1.1: Input Coefficient Matrix A␣
 ↪({len(A_matrix)} x {len(A_matrix[0])})")
print(f"\nOutput 1.2: Target Vector b ({len(b_vector)} x 1)\n  [ " + ", ".
 ↪join(f"{v:12.8f}" for v in b_vector) + " ]")

display_matrix(aug_matrix, f"Output 2: Augmented Matrix [A | b]␣
 ↪({len(aug_matrix)} x {len(aug_matrix[0])})")
display_matrix(ref_matrix, "Output 3: Row Echelon Form (REF)")
```

```
display_matrix(rref_matrix, "Output 4: Reduced Row Echelon Form (RREF)")

print(f"\nOutput 5: Column Classifications")
print(f"  Pivot Columns (Basic Variables)    : {p_cols}")
print(f"  Non-Pivot Columns (Free Variables) : {f_cols}")

print(f"\nOutput 6: Particular Solution (x_p)\n  [ " + ", ".join(f"{v:12.8f}"␣
 ↪for v in x_p) + " ]")

print("\nOutput 7: Homogeneous Basis Solutions (Ax = 0)")
for i, vec in enumerate(null_vectors):
    print(f"  Basis Vector {i+1} (For free column {f_cols[i]}): [ " + ", ".
 ↪join(f"{v:12.8f}" for v in vec) + " ]")

print("\nOutput 8: General Solution and Numerical Verification")
print(f"  Chosen Scalars for Free Variables    : {scalars[:len(f_cols)]}")
print(f"  Constructed General Solution (x_gen) :\n    [ " + ", ".join(f"{v:12.
 ↪8f}" for v in x_general) + " ]")
print(f"  Computed Result of A * x_gen         :\n    [ " + ", ".join(f"{v:12.
 ↪8f}" for v in Ax_calculated) + " ]")
print(f"  Expected Target Vector b             :\n    [ " + ", ".join(f"{v:12.
 ↪8f}" for v in b_vector) + " ]")
print(f"  Maximum Error ||Ax - b||_inf         : {max_residual:.14e}")

if max_residual < 1e-8:
    print("  Verification Status: SUCCESS (The residual error is strictly␣
 ↪within machine precision.)")
else:
    print("  Verification Status: FAILED")
```

```
Question 1 Part 3: Numerical Demonstration and Verification Outputs

Output 1.1: Input Coefficient Matrix A (5 x 7)
  [   2.45547483,   5.95458229,   2.04437936,   6.36480424,   7.81479851,
7.61242726,   7.59472851 ]
  [   3.53367692,   2.70265370,   5.41200135,   7.16081348,   2.93331039,
9.76861608,   8.44074333 ]
  [   5.18502027,   4.28060711,   3.79442279,   8.71577402,   5.64412617,
7.05044090,   4.77651330 ]
  [   6.18419761,   2.43794230,   7.60133957,   5.84500032,   3.48911361,
8.88905493,   5.16927402 ]
  [   2.92519780,   2.09576430,   8.19717364,   6.44851789,   5.08583219,
8.71404020,   5.12377680 ]

Output 1.2: Target Vector b (5 x 1)
  [   9.42294314,   5.92485419,   6.66565873,   7.90031563,   5.71559334 ]
```


ઉપરોક્ત કોડ અને આઉટપુટ.

```
Output 2: Augmented Matrix [A | b] (5 x 8)
  [   2.45547483,   5.95458229,   2.04437936,   6.36480424,   7.81479851,
7.61242726,   7.59472851,   9.42294314 ]
  [   3.53367692,   2.70265370,   5.41200135,   7.16081348,   2.93331039,
9.76861608,   8.44074333,   5.92485419 ]
  [   5.18502027,   4.28060711,   3.79442279,   8.71577402,   5.64412617,
7.05044090,   4.77651330,   6.66565873 ]
  [   6.18419761,   2.43794230,   7.60133957,   5.84500032,   3.48911361,
8.88905493,   5.16927402,   7.90031563 ]
  [   2.92519780,   2.09576430,   8.19717364,   6.44851789,   5.08583219,
8.71404020,   5.12377680,   5.71559334 ]

Output 3: Row Echelon Form (REF)
  [   6.18419761,   2.43794230,   7.60133957,   5.84500032,   3.48911361,
8.88905493,   5.16927402,   7.90031563 ]
  [   0.00000000,   4.98658186,  -0.97378066,   4.04400984,   6.42942385,
4.08297170,   5.54223552,   6.28607278 ]
  [   0.00000000,   0.00000000,   4.78572009,   2.91934506,   2.22011823,
3.73762916,   1.63102817,   0.79042618 ]
  [   0.00000000,   0.00000000,   0.00000000,   3.30799896,   0.82873874,
-0.56078592,  -1.31332166,  -2.42382871 ]
  [   0.00000000,   0.00000000,   0.00000000,   0.00000000,  -1.85205398,
2.91355559,   4.35472480,   0.97054363 ]

Output 4: Reduced Row Echelon Form (RREF)
  [   1.00000000,   0.00000000,   0.00000000,   0.00000000,   0.00000000,
-0.73226916,  -1.30775450,   0.17355904 ]
  [   0.00000000,   1.00000000,   0.00000000,   0.00000000,   0.00000000,
2.93325784,   4.24399835,   2.57538117 ]
  [   0.00000000,   0.00000000,   1.00000000,   0.00000000,   0.00000000,
1.37378471,   1.31443803,   0.77514753 ]
  [   0.00000000,   0.00000000,   0.00000000,   1.00000000,   0.00000000,
0.22458986,   0.19204582,  -0.60143292 ]
  [   0.00000000,   0.00000000,   0.00000000,   0.00000000,   1.00000000,
-1.57314831,  -2.35129475,  -0.52403636 ]

Output 5: Column Classifications
  Pivot Columns (Basic Variables)    : [0, 1, 2, 3, 4]
  Non-Pivot Columns (Free Variables) : [5, 6]

Output 6: Particular Solution (x_p)
  [   0.17355904,   2.57538117,   0.77514753,  -0.60143292,  -0.52403636,
0.00000000,   0.00000000 ]

Output 7: Homogeneous Basis Solutions (Ax = 0)
  Basis Vector 1 (For free column 5): [   0.73226916,  -2.93325784,
-1.37378471,  -0.22458986,   1.57314831,   1.00000000,   0.00000000 ]
  Basis Vector 2 (For free column 6): [   1.30775450,  -4.24399835,
```


확인함 잘했어요.

```
-1.31443803,  -0.19204582,   2.35129475,   0.00000000,   1.00000000 ]

Output 8: General Solution and Numerical Verification
  Chosen Scalars for Free Variables    : [3.14159265, -2.71828182]
  Constructed General Solution (x_gen) :
    [  -1.08079484,   4.89666345,   0.03228856,  -0.78496810,  -1.97332696,
3.14159265,  -2.71828182 ]
  Computed Result of A * x_gen         :
    [   9.42294314,   5.92485419,   6.66565873,   7.90031563,   5.71559334 ]
  Expected Target Vector b             :
    [   9.42294314,   5.92485419,   6.66565873,   7.90031563,   5.71559334 ]
  Maximum Error ||Ax - b||_inf         : 1.24344978758018e-14
  Verification Status: SUCCESS (The residual error is strictly within machine
precision.)
```

Q2) Consider a dataset $X \in \mathbb{R}^{500\times 6}$ constructed as follows: the first four features f_1, f_2, f_3, f_4 are generated as random features sampled from a standard normal distribution (read about this). The fifth and sixth features are defined by the relations:

$$f_5 = 2f_1 + 3f_2, \qquad f_6 = f_3 - 2f_4$$

Perform the following tasks in sequence:

1. Write a Python code to generate the dataset $X = [f_1, f_2, f_3, f_4, f_5, f_6]$.

[0.5 mark]

[4]:
```python
import numpy as np

# Fix the numpy random seed so the standard normal distribution generates
# the exact same values across different runs.
np.random.seed(42)

# Define the number of rows for our dataset based on the assignment description.
n_samples = 500

# Sample the first four features independently from a standard normal␣
 ↪distribution.
# As confirmed by Prof. Saurabh, using np.random.randn is fully permissible␣
 ↪here.
# Apply np.round to strictly enforce the 8-decimal format requirement.
f1 = np.round(np.random.randn(n_samples), 8)
f2 = np.round(np.random.randn(n_samples), 8)
f3 = np.round(np.random.randn(n_samples), 8)
f4 = np.round(np.random.randn(n_samples), 8)
```


रक्षेत्रा उमर्ड 2412.

```
# Calculate the fifth and sixth dependent features based on the exact linear␣
 ↪relations provided.
f5 = np.round(2.0 * f1 + 3.0 * f2, 8)
f6 = np.round(f3 - 2.0 * f4, 8)

# Stack the 1D arrays side-by-side to form the final 2D dataset matrix X.
X = np.column_stack((f1, f2, f3, f4, f5, f6))

print("Question 2 Task 1: Dataset Generation")
print(f"Matrix X Dimensions: {X.shape[0]} rows and {X.shape[1]} columns")

print("\nFirst 10 Rows of Dataset X (Formatted to 8 Decimal Places):")
for i in range(10):
    row_str = ", ".join(f"{val:12.8f}" for val in X[i])
    print(f"  Row {i+1:02d}: [ {row_str} ]")
```

```
Question 2 Task 1: Dataset Generation
Matrix X Dimensions: 500 rows and 6 columns

First 10 Rows of Dataset X (Formatted to 8 Decimal Places):
  Row 01: [   0.49671415,   0.92617755,   1.39935544,   0.77836108,
3.77196095,  -0.15736672 ]
  Row 02: [  -0.13826430,   1.90941664,   0.92463368,  -0.55118572,
5.45172132,   2.02700512 ]
  Row 03: [   0.64768854,  -1.39856757,   0.05963037,  -0.81819888,
-2.90032563,   1.69602813 ]
  Row 04: [   1.52302986,   0.56296924,  -0.64693678,  -0.00337446,
4.73496744,  -0.64018786 ]
  Row 05: [  -0.23415337,  -0.65064257,   0.69822331,  -0.17018462,
-2.42023445,   1.03859255 ]
  Row 06: [  -0.23413696,  -0.48712538,   0.39348539,  -0.45322805,
-1.92965006,   1.29994149 ]
  Row 07: [   1.57921282,  -0.59239392,   0.89519322,   0.69638745,
1.38124388,  -0.49758168 ]
  Row 08: [   0.76743473,  -0.86399077,   0.63517180,   0.95530521,
-1.05710285,  -1.27543862 ]
  Row 09: [  -0.46947439,   0.04852163,   1.04955272,   0.08840689,
-0.79338389,   0.87273894 ]
  Row 10: [   0.54256004,  -0.83095012,  -0.53523521,   1.47753008,
-1.40773028,  -3.49029537 ]
```

2. Write a Python code which computes the rank of X and display the output for the dataset X generated in step 1.

[0.5 mark]

[5]:

```python
# As per Prof. Saurabh's explicit instruction, we run the REF function created
# in Question 1 to compute the rank by counting the resulting pivot rows, rather than
# rewriting the elimination logic from scratch.

# Convert the numpy array X into a standard Python list of lists
# so it is compatible with the Q1 function.
X_list = X.tolist()

# Call the Q1 compute_ref function. We pass is_augmented=False because
# X is a standard dataset matrix, not a system of equations with an appended target vector.
ref_X = compute_ref(X_list, is_augmented=False)

# The rank is the number of rows in the REF matrix that are not entirely zeros.
rank_X = 0
for row in ref_X:
    # If any value in the row is greater than our near-zero tolerance, it is a valid pivot row.
    if any(abs(val) > 1e-10 for val in row):
        rank_X += 1

print("Question 2 Task 2: Rank Computation")
print(f"\nComputed Rank of Dataset Matrix X : {rank_X}")
print(f"Theoretical Expected Rank         : 4")

print("\nMathematical Reasoning:")
print("Because features f5 and f6 are exact linear combinations of f1, f2, f3, and f4,")
print("they do not contribute any new, independent dimensions. Therefore, the matrix")
print(f"spans a {rank_X}-dimensional subspace, making its rank exactly {rank_X}.")
```

```
Question 2 Task 2: Rank Computation

Computed Rank of Dataset Matrix X : 4
Theoretical Expected Rank         : 4

Mathematical Reasoning:
Because features f5 and f6 are exact linear combinations of f1, f2, f3, and f4,
they do not contribute any new, independent dimensions. Therefore, the matrix
spans a 4-dimensional subspace, making its rank exactly 4.
```

3. Numerical Experiment with the Power Method

Read about the power method for finding the dominant eigenvalue and its corresponding eigenvector and perform the following tasks.

(a) Write a Python code to compute the covariance matrix

$$C = \frac{1}{n} X^T X$$

where n is number of data points.

[0.5 mark]

[6]:
```
def compute_covariance_matrix(data_mat):
    """
    Computes the empirical covariance matrix mathematically using matrix␣
 ↪multiplication.
    """
    n = data_mat.shape[0]

    # Transpose X, multiply it by X, and scale the result by 1/n.
    return (1.0 / n) * np.dot(data_mat.T, data_mat)

# Execute the function on the dataset to obtain the covariance matrix C.
C_matrix = compute_covariance_matrix(X)

print("Question 2 Task 3(a): Covariance Matrix Computation")
print(f"\nCovariance Matrix C ({C_matrix.shape[0]} x {C_matrix.shape[1]}):")
for row in C_matrix:
    print("  [ " + ", ".join(f"{val:12.8f}" for val in row) + " ]")
```

```
Question 2 Task 3(a): Covariance Matrix Computation

Covariance Matrix C (6 x 6):
  [   0.96097898,  -0.07225555,  -0.05643179,   0.06203769,   1.70519131,
-0.18050717 ]
  [  -0.07225555,   0.95557846,   0.07842969,  -0.01966945,   2.72222428,
0.11776860 ]
  [  -0.05643179,   0.07842969,   1.03032538,  -0.01828330,   0.12242549,
1.06689197 ]
  [   0.06203769,  -0.01966945,  -0.01828330,   0.96755081,   0.06506701,
-1.95338491 ]
  [   1.70519131,   2.72222428,   0.12242549,   0.06506701,  11.57705545,
-0.00770854 ]
  [  -0.18050717,   0.11776860,   1.06689197,  -1.95338491,  -0.00770854,
4.97366179 ]
```

(b) Implement the Power Method in Python to approximate the largest eigenvalue λ_1 and its corresponding eigenvector v_1 of C. Show the code and the outputs.

[1 mark]

[7]:
```
def power_method_single(A, max_iter=2000, tol=1e-12):
    """
    Iteratively estimates the largest eigenvalue and its corresponding␣
 ↪eigenvector
    using the Power Method and the Rayleigh quotient.
    """
    n = A.shape[0]

    # Initialize the starting vector with 1s and normalize it to unit length.
    v = np.ones(n) / np.sqrt(n)
    prev_lambda = 0.0

    for it in range(1, max_iter + 1):
        # Multiply the matrix by the current vector estimate.
        w = np.dot(A, v)

        # As per Prof. Saurabh's guidance, calculate the magnitude using np.
 ↪sqrt and np.sum
        # instead of using the built-in np.linalg.norm function.
        norm_w = np.sqrt(np.sum(w**2))

        # If the norm is practically zero, it indicates we have reached a null␣
 ↪space.
        if norm_w < 1e-14:
            return 0.0, np.zeros(n), it

        v_next = w / norm_w

        # Compute the Rayleigh quotient to estimate the current eigenvalue.
        curr_lambda = float(np.dot(v_next.T, np.dot(A, v_next)))

        # Stop iterating once the change in the eigenvalue estimate falls below␣
 ↪the tolerance.
        if abs(curr_lambda - prev_lambda) < tol:
            return curr_lambda, v_next, it

        # Prepare the variables for the next iteration step.
        prev_lambda = curr_lambda
        v = v_next

    return prev_lambda, v, max_iter

# Apply the Power Method to the Covariance matrix C to find the dominant␣
 ↪eigenpair.
lambda_1, v_1, iters_1 = power_method_single(C_matrix)

print("Question 2 Task 3(b): Dominant Eigenpair via Power Method\n")
```


শ্রেয়সী কুমার 2412

```
print(f"Dominant Eigenvalue (lambda_1)    : {lambda_1:14.8f}")
print("Dominant Eigenvector (v_1)        : [ " + ", ".join(f"{val:10.8f}" for␣
 ↪val in v_1) + " ]")
print(f"Iterations to Converge (tol 1e-12): {iters_1}")
```

```
Question 2 Task 3(b): Dominant Eigenpair via Power Method

Dominant Eigenvalue (lambda_1)    :    12.47020120
Dominant Eigenvector (v_1)        : [ 0.14131835, 0.22695920, 0.01108610,
0.00594237, 0.96351429, -0.00079864 ]
Iterations to Converge (tol 1e-12): 21
```

(c) Write a Python code to obtain the next largest eigenvalue λ_2 and its corresponding eigenvector v_2 by applying power method on $C - v_1 v_1^T C$. Having found out $v_1, v_2, \ldots, v_{k-1}$, one can find λ_k and its corresponding eigenvector v_k by applying power method on:

$$C - \sum_{j=1}^{k-1} v_j v_j^T C$$

Give the code and also display the obtained eigenvalues and the corresponding eigenvectors.

[1.5 marks]

[8]:
```
def extract_all_eigenpairs_deflation(C_mat, total_components):
    """
    Finds all eigenvalues and eigenvectors sequentially by applying Hotelling␣
 ↪Deflation.
    It removes the variance of previously found eigenvectors from the matrix at␣
 ↪each step.
    """
    eigenvalues = []
    eigenvectors = []
    iteration_counts = []

    for k in range(total_components):
        # Always begin the deflation process from the original matrix C.
        C_k = C_mat.copy()

        # Deflate the matrix using all valid eigenvectors found so far.
        for j in range(k):
            # Explicitly only deflate non-zero subspaces to avoid amplifying␣
 ↪floating-point noise.
            if abs(eigenvalues[j]) > 1e-10:
                v_j = eigenvectors[j].reshape(-1, 1)

                # Apply the deflation formula: (v_j * v_j^T) * C
                deflation_term = np.dot(np.dot(v_j, v_j.T), C_mat)
                C_k -= deflation_term
```

```
        # Apply the Power Method to the newly deflated matrix to find the next␣
 ↪component.
        lam, vec, iters = power_method_single(C_k)

        # If the eigenvalue is negligible, force it to exactly zero to␣
 ↪accurately reflect the true rank.
        if abs(lam) < 1e-10:
            lam = 0.0
            vec = np.zeros(C_mat.shape[0])

        eigenvalues.append(lam)
        eigenvectors.append(vec)
        iteration_counts.append(iters)

    return np.array(eigenvalues), np.array(eigenvectors), np.
 ↪array(iteration_counts)

# Extract all eigenpairs dynamically based on the dimensions of the matrix.
total_dims = C_matrix.shape[0]
def_lambdas, def_vectors, def_iters =␣
 ↪extract_all_eigenpairs_deflation(C_matrix, total_dims)

print("Question 2 Task 3(c): All Eigenpairs via Hotelling Deflation")
for k in range(total_dims):
    print(f"\n--- Component {k+1} ---")
    print(f"  Eigenvalue (lambda_{k+1}) : {def_lambdas[k]:14.8f}")
    print(f"  Eigenvector (v_{k+1})     : [ " + ", ".join(f"{v:10.8f}" for v in␣
 ↪def_vectors[k]) + " ]")
    print(f"  Iteration Count           : {def_iters[k]}")

print("\nObservation:")
print(f"Because the computed rank of the dataset is {rank_X}, the covariance␣
 ↪matrix has exactly {rank_X} positive eigenvalues.")
print(f"The remaining {total_dims - rank_X} eigenvalues evaluate to 0.0,␣
 ↪correctly reflecting the zero null space.")
```

```
Question 2 Task 3(c): All Eigenpairs via Hotelling Deflation

--- Component 1 ---
  Eigenvalue (lambda_1) :    12.47020120
  Eigenvector (v_1)     : [ 0.14131835, 0.22695920, 0.01108610, 0.00594237,
0.96351429, -0.00079864 ]
  Iteration Count           : 21

--- Component 2 ---
  Eigenvalue (lambda_2) :     5.98130177
```

```
  Eigenvector (v_2)    : [ -0.03971941, 0.02655903, 0.19865413, -0.35651100,
0.00023828, 0.91167613 ]
  Iteration Count          : 10

--- Component 3 ---
  Eigenvalue (lambda_3) :     1.05483099
  Eigenvector (v_3)    : [ -0.63152951, 0.41635267, 0.57452683, 0.30921806,
-0.01400101, -0.04390929 ]
  Iteration Count          : 132

--- Component 4 ---
  Eigenvalue (lambda_4) :     0.95881690
  Eigenvector (v_4)    : [ 0.54213254, -0.36273607, 0.68093017, 0.33253890,
-0.00394313, 0.01585237 ]
  Iteration Count          : 2

--- Component 5 ---
  Eigenvalue (lambda_5) :     0.00000000
  Eigenvector (v_5)    : [ 0.00000000, 0.00000000, 0.00000000, 0.00000000,
0.00000000, 0.00000000 ]
  Iteration Count          : 1

--- Component 6 ---
  Eigenvalue (lambda_6) :     0.00000000
  Eigenvector (v_6)    : [ 0.00000000, 0.00000000, 0.00000000, 0.00000000,
0.00000000, 0.00000000 ]
  Iteration Count          : 1

Observation:
Because the computed rank of the dataset is 4, the covariance matrix has exactly
4 positive eigenvalues.
The remaining 2 eigenvalues evaluate to 0.0, correctly reflecting the zero null
space.
```

(d) Find all the eigenvalues and eigenvectors using Python function and compare with the obtained result in (c).

[0.5 mark]

[9]:
```
# As advised by Prof. Saurabh, use 'eigh' because our covariance matrix C is␣
 ↪symmetric.
# This function is optimized to return real eigenvalues and orthogonal␣
 ↪eigenvectors.
true_evals_raw, true_evecs_raw = np.linalg.eigh(C_matrix)

# The 'eigh' function returns eigenvalues in ascending order.
# Sort them in descending order to properly compare them against the Power␣
 ↪Method results.
```

```
sort_idx = np.argsort(true_evals_raw)[::-1]
true_lambdas = true_evals_raw[sort_idx]

print("Question 2 Task 3(d): Benchmark against numpy.linalg.eigh\n")
print(f"{'Component':^11} | {'Power Method Lambda':^22} | {'NumPy Actual␣
 ↪Lambda':^22} | {'Absolute Error':^18}")
print("-" * 82)

for k in range(len(true_lambdas)):
    # Calculate the absolute error between the scratch method and the built-in␣
 ↪library.
    abs_err = abs(def_lambdas[k] - true_lambdas[k])
    print(f"  lambda_{k+1}  | {def_lambdas[k]:22.8f} | {true_lambdas[k]:22.8f}␣
 ↪| {abs_err:18.4e}")

print("-" * 82)
print("Conclusion: All power method eigenvalues match the numpy benchmark␣
 ↪precisely.")
```

```
Question 2 Task 3(d): Benchmark against numpy.linalg.eigh

 Component  |  Power Method Lambda   |  NumPy Actual Lambda   |   Absolute Error
----------------------------------------------------------------------------------
--
  lambda_1  |           12.47020120 |           12.47020120 |
7.6383e-14
  lambda_2  |            5.98130177 |            5.98130177 |
7.1942e-14
  lambda_3  |            1.05483099 |            1.05483099 |
4.2690e-12
  lambda_4  |            0.95881690 |            0.95881690 |
4.2716e-12
  lambda_5  |            0.00000000 |           -0.00000000 |
2.2081e-16
  lambda_6  |            0.00000000 |           -0.00000000 |
5.8410e-16
----------------------------------------------------------------------------------
--
Conclusion: All power method eigenvalues match the numpy benchmark precisely.
```

(e) Compare the number of iterations required to get an accuracy of 10^{-7} using the power method. The actual values can be taken as the one obtained in (d).

[0.5 mark]

[10]:
```
def check_iterations_to_accuracy(C_mat, actual_lambdas, target_accuracy=1e-7,␣
 ↪max_iterations=5000):
    """
```

```
    Executes the power method step-by-step to record the exact iteration count
    needed for the estimate to reach within 10^-7 of the actual numpy␣
↪eigenvalue.
    """
    results = []
    found_vectors = []

    for k in range(len(actual_lambdas)):
        actual_val = actual_lambdas[k]

        # Tracking iterations is not mathematically applicable for the zero␣
↪subspace components.
        if abs(actual_val) < 1e-10:
            results.append((k + 1, actual_val, 0.0, 0, "N/A (Zero Subspace)"))
            continue

        # Construct the deflated matrix for the current step.
        C_k = C_mat.copy()
        for j in range(k):
            v_j = found_vectors[j].reshape(-1, 1)
            C_k -= np.dot(np.dot(v_j, v_j.T), C_mat)

        # Set up the starting variables for the power method loop.
        n = C_k.shape[0]
        v = np.ones(n) / np.sqrt(n)
        iters_needed = max_iterations
        achieved_lam = 0.0
        final_err = 0.0

        for it in range(1, max_iterations + 1):
            w = np.dot(C_k, v)

            # Using np.sqrt and np.sum for vector magnitude as permitted by␣
↪Prof. Saurabh.
            norm_w = np.sqrt(np.sum(w**2))

            # Normalize the vector to feed into the next step.
            v = w / norm_w

            # Calculate the Rayleigh quotient estimate for the current loop.
            curr_lam = float(np.dot(v.T, np.dot(C_k, v)))

            # Check the absolute difference against the TRUE numpy benchmark␣
↪value.
            error = abs(curr_lam - actual_val)
```

```
            # If the error falls below the threshold, record the iteration␣
 ↪count and break.
            if error <= target_accuracy:
                iters_needed = it
                achieved_lam = curr_lam
                final_err = error
                break

            achieved_lam = curr_lam
            final_err = error

        found_vectors.append(v)
        results.append((k + 1, actual_val, achieved_lam, iters_needed,␣
 ↪f"{final_err:.4e}"))

    return results

# Run the convergence analysis.
accuracy_report = check_iterations_to_accuracy(C_matrix, true_lambdas,␣
 ↪target_accuracy=1e-7)

print("Question 2 Task 3(e): Iterations Required for 10^-7 Accuracy\n")
print(f"{'Component':^11} | {'Actual Value':^15} | {'Achieved Value':^16} |␣
 ↪{'Iterations':^12} | {'Final Error':^15}")
print("-" * 77)

for record in accuracy_report:
    idx, actual, achieved, iters, err_str = record
    if iters > 0:
        print(f"  lambda_{idx:<2} | {actual:15.8f} | {achieved:16.8f} | {iters:
 ↪^12} | {err_str:^15}")
    else:
        print(f"  lambda_{idx:<2} | {actual:15.8f} | {'0.00000000':^16} |␣
 ↪{'Converged':^12} | {err_str:^15}")
print("-" * 77)
print("\nConclusion: The number of iterations required depends on the 'spectral␣
 ↪gap'.")
print("Larger distances between subsequent eigenvalues lead to faster geometric␣
 ↪convergence.")
```

```
Question 2 Task 3(e): Iterations Required for 10^-7 Accuracy

 Component  |  Actual Value   |  Achieved Value  |  Iterations  |   Final Error
-----------------------------------------------------------------------------
  lambda_1  |     12.47020120 |      12.47020116 |      12      |   4.3095e-08
  lambda_2  |      5.98130177 |       5.98130181 |      6       |   3.6238e-08
  lambda_3  |      1.05483099 |       1.05483091 |      80      |   8.4280e-08
```


বারবার 3য়ার আর.

```
  lambda_4  |      0.95881690 |       0.95881699 |      1       |   9.1137e-08
  lambda_5  |     -0.00000000 |    0.00000000    |  Converged   | N/A (Zero
Subspace)
  lambda_6  |     -0.00000000 |    0.00000000    |  Converged   | N/A (Zero
Subspace)
-------------------------------------------------------------------------------

Conclusion: The number of iterations required depends on the 'spectral gap'.
Larger distances between subsequent eigenvalues lead to faster geometric
convergence.
```


વિરગીયા. ડુબક ડુબક.